

Fault Tolerance in the Extended Automated Bottling System

Individual Project Interim Report - Christian Motta

1 Goals

The core ABS has no way of noticing when something inside a station has gone wrong, and no way of continuing production if it does. If a valve opens and a fill never reaches its target, the controller just keeps waiting on `levelAtTarget` forever, and the whole line sits stopped until someone notices, walks over and fixes it. Detecting a fault and stopping safely is only half the job. What makes a fault tolerated rather than just noticed is that production keeps going while the broken part gets fixed, which means the design needs a way to route around a faulted station, not just stop at one.

This IP has two goals. First, detect faults locally, inside the station where they happen, using signals and timing that the station's own controller already has, with no new sensors. Second, once a fault is detected, take the faulted station out of rotation, flag it for an operator, and let production continue rather than stall on it. I am working the design out first against the Filling Unit, since it is the only station our group has modelled so far, but the final detection and flagging mechanism will include the Filling Unit, Lid Placer, and Capper.

2 Background and Literature

SystemJ structures the system as separate clock domains, each internally synchronous and only able to affect another domain through the signals it explicitly emits (Malik, Salcic, Roop & Girault, 2010). That matters here for a specific reason. A fault inside one controller's clock domain cannot corrupt another domain's state by accident, it can only become visible to the rest of the system through a signal that domain chooses to send. A station's fault detection can therefore live entirely inside that station's own controller, and the only thing crossing the boundary to the rest of the line is a fault signal.

The redundancy side is closer to the SOSJ framework work of Dwi Atmojo, Salcic, and Wang (2018), which addresses a similar problem in manufacturing systems generally. A machine's software behaviour needs to change when the physical process it controls changes, without resetting the whole operation to do it. Their motivation is that existing manufacturing control software has limited ability to reconfigure at runtime. Taking a faulted station out of rotation and routing around it is a small instance of the same idea, a runtime change in which station handles a bottle, made without stopping the rest of the line.

3 Idea and Methodology

Detection stays local to the controller and easy, but the two kinds of station this applies to need different detection styles because they fail differently. The Filling Unit's fault is a continuous condition, a valve left open.

`FillerControllerCD` currently runs two parallel threads, the dosing loop and the thread that emits `fillDoneE`. I am adding a third thread that counts reactions while `valve1Open` or `valve2Open` is held open without `levelAtTarget` arriving, and raises an internal `fillFault` signal once that count passes a fixed threshold. The same thread also checks the reported fill level against `doseTargetMl` and raises the same signal if it exceeds the target.

Lid Placement and Capper fail differently. Both act on a discrete physical object, a lid, that can be knocked loose or flung off mid-action rather than simply not arriving on time. A single failed attempt there is not necessarily a real fault, so a plain timeout is the wrong tool. Instead these two stations retry the action, gripping and placing the lid or clamping and twisting it, a fixed number of times, three attempts as a starting point, before declaring a fault. The timeout still applies within each attempt, in case an attempt itself hangs, but the fault is only raised once the retry count is exhausted.

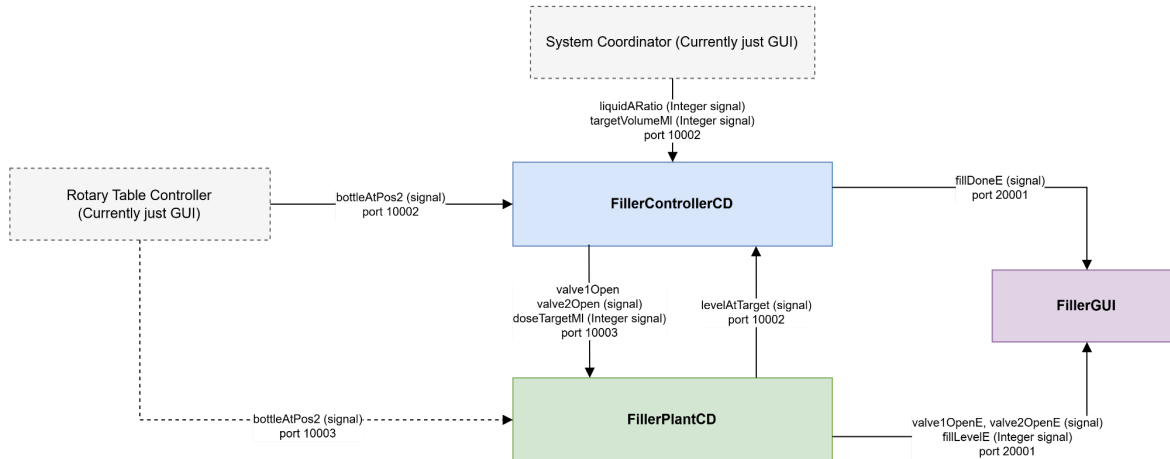


Figure 1. Example of fault detection extension across the Filling Unit.

Once a fault signal fires, it does two things locally before it goes anywhere else. It forces the station's actuators into a safe state, valves closed for the Filler, gripper retracted for Lid Placement and Capper, and it emits a signal to that station's GUI so the fault is visible to an operator rather than the bottle just sitting there unexplained. The fault also escalates as a station status signal, fillFaultOut for the Filler and the equivalent for Lid Placement and Capper, which the System Coordinator consumes to reroute the bottle to a redundant instance of that station rather than simply withholding bottles from the faulted one, described in Section 6.

4 Integration into the Group Project

This fits directly into the mixed control strategy our group already argued for in the GP report. Station-to-station handoffs stay fast and local, but which physical instance of a station, primary or redundant, a bottle should go to next is exactly the kind of system-wide decision already associated with the System Coordinator, alongside batch tracking and sort destination, and our GP report's Control Strategy section also names fault response as one of the decisions the Coordinator has authority over. The fault-out signals and the redundant-routing decision they trigger are simply another input to that existing authority.

5 Scope and Assumptions

Fault detection and mitigation in this design apply only to the Filling Unit, Lid Placement, and Capper, not to the conveyor or the turntable's rotation. Those three stations act on one bottle at a time from beside the line, so a faulted bottle can be rerouted to a redundant instance without disturbing anything else on the path. A conveyor or turntable fault affects every bottle currently on that path at once, not a single workpiece, so the same per-bottle rerouting logic does not apply, and is out of scope for this design.

Only one station is assumed faulted at a time, since handling simultaneous faults across multiple stations is a substantially harder routing problem to solve. Detection thresholds, timeout length and retry count, are fixed constants tuned against the simulated behaviour of each station rather than something reconfigured per batch. Each station's GUI includes a way to manually trigger a fault for testing, so the detection and rerouting path can be demonstrated without waiting for a real failure, and a separate way to clear a fault once an operator has fixed the station, returning it to rotation, since the controller has no way of confirming a physical repair actually happened on its own.

Not every stall counts as a fault. A retry that succeeds within the allowed count, a station sitting idle because no bottle has arrived yet, and a manual override in progress are all normal operation, and none of these will raise fault signals.

6 Redundancy and Rerouting

Redundancy is provided as a shared duplicate line rather than a separate redundant instance of each station individually. The turntable layout has a vacant slot alongside its five active positions. A bottle that faults at any of the three stations continues past the collection point, and a robot arm transfers it into the vacant slot of a second, parallel turntable setup with its own Filling Unit, Lid Placement, and Capper, which then continues that bottle's processing from wherever it left off. One shared arm and one shared duplicate line cover faults from all three stations, rather than needing a separate redundant instance and transport path for each.

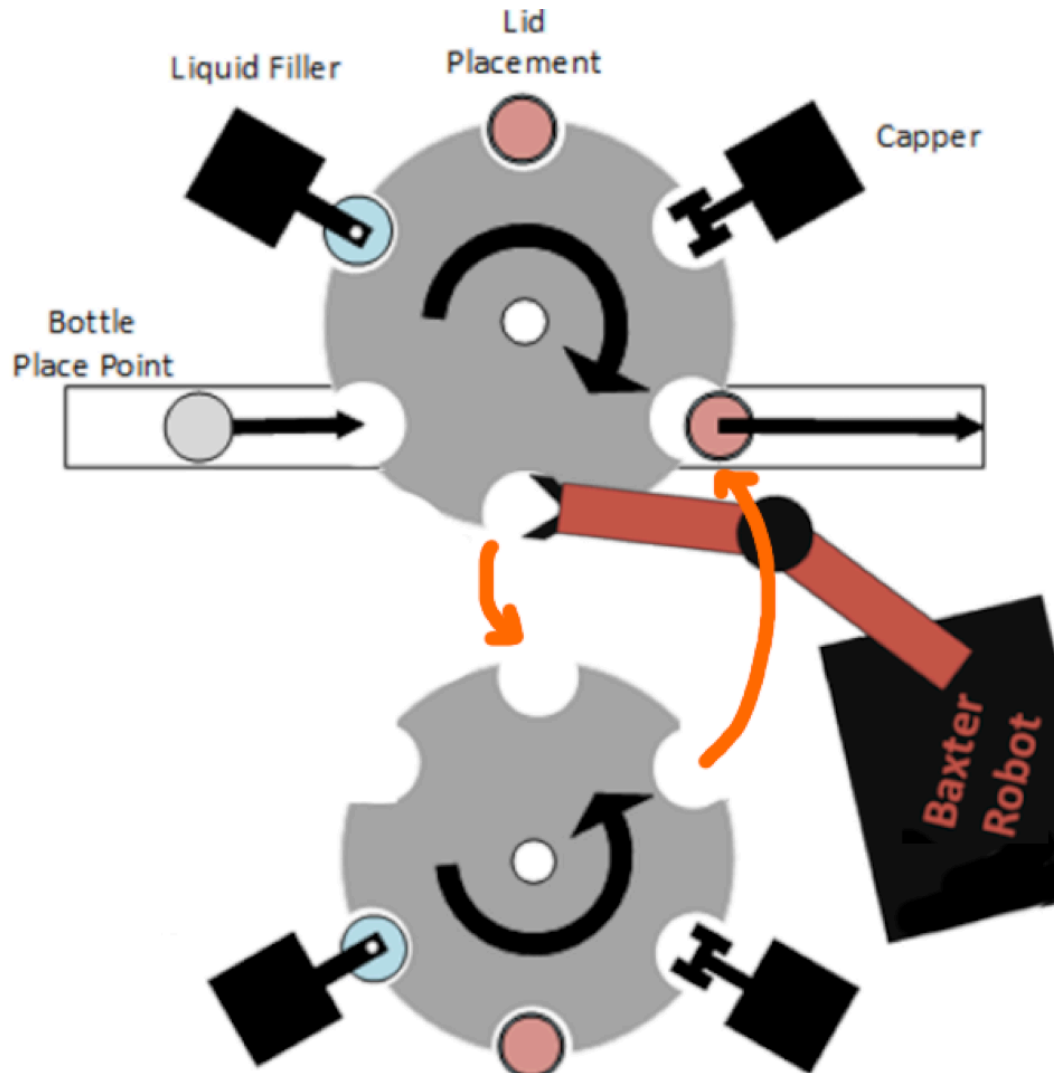


Figure 2. Vacant turntable slot as a robot-arm transfer point into a parallel duplicate line..

A bottle only counts as an unrecoverable, rejected product if it triggers the same fault criteria a second time, on the redundant machine, since there is no further fallback beyond that. A bottle that faults once and then completes successfully on the redundant machine is not treated as a faulted product, only the original station is.

7 Milestone 2 Plan

The next steps are implementing the fault detection threads and their signals for the Filling Unit first, then extending the same structure to Lid Placement and Capper once those stations exist, and adding a fault indicator and manual fault trigger to the GUI for each station. The redundant line and its vacant-slot handoff mechanism are implemented alongside this, once the primary line's three stations exist to duplicate.